OPERATING SYSTEMS – ASSIGNMENT 1

24k-0812

Laiba Jamil

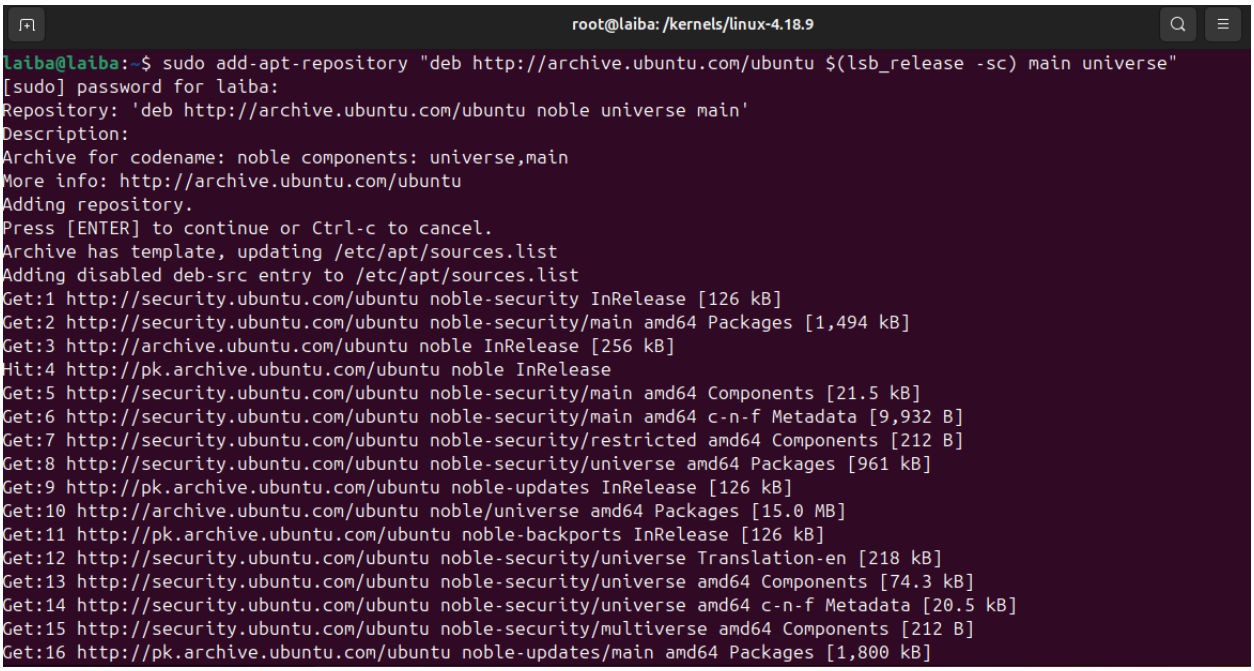
BCS – 4F

CONFIGURING NEW KERNEL

1. Downloading Dependencies

Commands:

$ sudo add-apt-repository "deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc) main universe"
$ sudo apt-get update
$ sudo apt-get install libncurses5-dev



```
root@laiba: /kernels/linux-4.18.9
laiba@laiba:~$ sudo add-apt-repository "deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc) main universe"
[sudo] password for laiba:
Repository: 'deb http://archive.ubuntu.com/ubuntu noble universe main'
Description:
Archive for codename: noble components: universe,main
More info: http://archive.ubuntu.com/ubuntu
Adding repository.
Press [ENTER] to continue or Ctrl-c to cancel.
Archive has template, updating /etc/apt/sources.list
Adding disabled deb-src entry to /etc/apt/sources.list
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,494 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9,932 B]
Get:7 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:8 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [961 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:11 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [218 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.3 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.5 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:16 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,800 kB]
```

```
root@laiba: /kernels/linux-4.18.9
laiba@laiba:~$ sudo apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
laiba@laiba:~$ sudo apt-get install libncurses5-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'libncurses-dev' instead of 'libncurses5-dev'
Suggested packages:
  ncurses-doc
The following NEW packages will be installed:
  libncurses-dev
0 upgraded, 1 newly installed, 0 to remove and 164 not upgraded.
Need to get 384 kB of archives.
After this operation, 2,417 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libncurses-dev amd64 6.4+20240113-1ubuntu2 [384 kB]
Fetched 384 kB in 17s (22.1 kB/s)
Selecting previously unselected package libncurses-dev:amd64.
(Reading database ... 154531 files and directories currently installed.)
Preparing to unpack .../libncurses-dev_6.4+20240113-1ubuntu2_amd64.deb ...
Unpacking libncurses-dev:amd64 (6.4+20240113-1ubuntu2) ...
Setting up libncurses-dev:amd64 (6.4+20240113-1ubuntu2) ...
```

CDK is a library written in C that provides a collection of widgets for text user interfaces (TUI) development. The widgets wrap n-curses functionality to make writing full screen curses programs faster.

2. OpenSSH/OpenSSL:

   Commands:

   $ sudo apt-get install libssl-dev

```
laiba@laiba:~$ sudo apt-get install libssl-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  libssl-doc
The following NEW packages will be installed:
  libssl-dev
0 upgraded, 1 newly installed, 0 to remove and 164 not upgraded.
Need to get 2,407 kB of archives.
After this operation, 13.3 MB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 libssl-dev amd64 3.0.13-0ubuntu3.7 [2,407 kB]
Fetched 2,407 kB in 11s (228 kB/s)
Selecting previously unselected package libssl-dev:amd64.
(Reading database ... 154612 files and directories currently installed.)
Preparing to unpack .../libssl-dev_3.0.13-0ubuntu3.7_amd64.deb ...
Unpacking libssl-dev:amd64 (3.0.13-0ubuntu3.7) ...
Setting up libssl-dev:amd64 (3.0.13-0ubuntu3.7) ...
```

OpenSSL is a robust, commercial-grade, and full-featured toolkit for the Transport Layer

Security (TLS) and Secure Sockets Layer (SSL) protocols. It is also a general-purpose cryptography library.

3. Downloading Kernel:

   Commands:

   $ sudo -i
   $ mkdir /kernels
   $ cd /kernels

```
laiba@laiba:~$ sudo -i
root@laiba:~# mkdir /kernels
root@laiba:~# cd /kernels
```

Go inside the root directory and create kernels then go inside kernels.

/kernels$ wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.8.tar.xz

```
root@laiba:/# cd /kernels
root@laiba:/kernels# wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.8.tar.xz
--2026-03-03 19:22:49--  https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.8.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 199.232.81.176, 2a04:4e42:54::432
Connecting to cdn.kernel.org (cdn.kernel.org)|199.232.81.176|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 142502100 (136M) [application/x-xz]
Saving to: ‘linux-6.8.tar.xz’

linux-6.8.tar.xz                100%[=====================================================>] 135.90M  1.76MB/s    in 92s

2026-03-03 19:24:22 (1.47 MB/s) - ‘linux-6.8.tar.xz’ saved [142502100/142502100]

root@laiba:/kernels#
```

/kernels$ tar xf linux-6.8.tar.xz
/kernels$ cd linux-6.8

```
root@laiba:/kernels# tar xf linux-6.8.tar.xz
root@laiba:/kernels# cd linux-6.8
root@laiba:/kernels/linux-6.8#
```

Downloaded Linux kernel 6.8 source code from kernel.org and extracted it into /kernels directory.

3. Configuring Kernel in Linux System

   Snapshot of the current state before further configuration.

```
root@laiba:/kernels/linux-6.8# cp /boot/config-$(uname -r) .config
root@laiba:/kernels/linux-6.8# ls .config
.config
root@laiba:/kernels/linux-6.8# ls
arch    COPYING  Documentation  include   ipc      kernel    MAINTAINERS  net     samples   sound  virt
block   CREDITS  drivers        init      Kbuild   lib       Makefile     README  scripts   tools
certs   crypto   fs             io_uring  Kconfig  LICENSES  mm           rust    security  usr
```

4. Copy Old Config File to Our New Kernel Folder

   Commands:

   $ ls /boot | grep config
   $ cp /boot/config-$(uname -r) .config

```
root@laiba:/kernels/linux-6.8# ls /boot | grep config
config-6.17.0-14-generic
root@laiba:/kernels/linux-6.8# cp /boot/config-$(uname -r) .config
root@laiba:/kernels/linux-6.8# ls -l .config
-rw-r--r-- 1 root root 302820 Mar  3 19:41 .config
root@laiba:/kernels/linux-6.8#
```

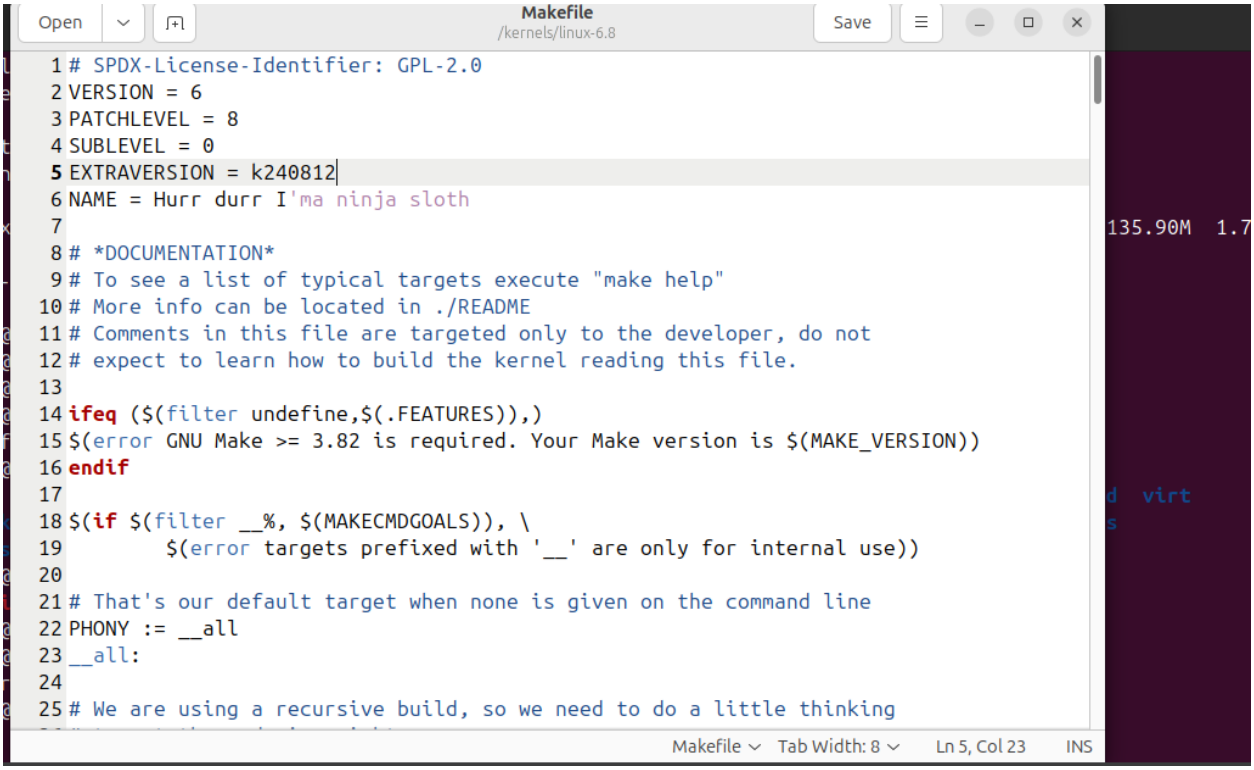
Copied the current running kernel configuration file to use as base configuration for the new kernel.

5. Naming the Kernel:

   Command:

   /kernels/linux-6.8 $ gedit Makefile

```
root@laiba:/kernels/linux-6.8# gedit Makefile
```



Modified the Makefile to change EXTRAVERSION so that the new kernel displays my roll number (24k-0812) when uname -r is executed.

6. Setting Configuration Parameter

   Command:

   /kernels/linux-6.8 $ yes ‘’ | make oldconfig -j4

```
  Perform selftest on priority array manager (TEST_PARMAN) [N/m/?] n
  Test module loading with 'hello world' module (TEST_LKM) [N/m/?] n
  Test module for compilation of bitops operations (TEST_BITOPS) [N/m/?] n
  Test module for stress/performance analysis of vmalloc allocator (TEST_VMALLOC) [N/m/?] n
  Test user/kernel boundary protections (TEST_USER_COPY) [N/m/?] (NEW)
  Test BPF filter functionality (TEST_BPF) [M/n/?] m
  Test blackhole netdev functionality (TEST_BLACKHOLE_DEV) [N/m/?] (NEW)
  Test find_bit functions (FIND_BIT_BENCHMARK) [N/m/y/?] n
  Test firmware loading via userspace interface (TEST_FIRMWARE) [N/m/y/?] n
  sysctl test driver (TEST_SYSCTL) [N/m/y/?] n
  udelay test driver (TEST_UDELAY) [N/m/y/?] n
  Test static keys (TEST_STATIC_KEYS) [N/m/?] n
  Test DYNAMIC_DEBUG (TEST_DYNAMIC_DEBUG) [N/m/y/?] n
  kmod stress tester (TEST_KMOD) [N/m/?] n
  Test memcat_p() helper function (TEST_MEMCAT_P) [N/m/y/?] n
  Test livepatching (TEST_LIVEPATCH) [N/m/?] (NEW)
  Perform selftest on object aggreration manager (TEST_OBJAGG) [N/m/?] n
  Test heap/page initialization (TEST_MEMINIT) [N/m/y/?] n
  Test HMM (Heterogeneous Memory Management) (TEST_HMM) [N/m/y/?] n
  Test freeing pages (TEST_FREE_PAGES) [N/m/y/?] n
  Test floating point operations in kernel space (TEST_FPU) [N/m/y/?] n
  Test clocksource watchdog in kernel space (TEST_CLOCKSOURCE_WATCHDOG) [N/m/y/?] n
  Test module for correctness and stress of objpool (TEST_OBJPOOL) [N/m/?] n
#
# configuration written to .config
#
```

Set configuration for .config file

/kernels/linux-6.8 $ make clean -j4

```
root@laiba:/kernels/linux-6.8

laiba@laiba:~$ make clean
```

Cleaned previous build files to ensure fresh compilation.

/kernel/linux-6.8 $ make bzImage -j4

```
root@laiba: /kernels/linux-6.8
  AS      arch/x86/boot/compressed/mem_encrypt.o
  CC      arch/x86/boot/compressed/pgtable_64.o
  CC      arch/x86/boot/compressed/sev.o
  CC      arch/x86/boot/compressed/acpi.o
  CC      arch/x86/boot/compressed/tdx.o
  AS      arch/x86/boot/compressed/tdcall.o
  CC      arch/x86/boot/compressed/tdx-shared.o
  CC      arch/x86/boot/compressed/mem.o
  CC      arch/x86/boot/compressed/efi.o
  AS      arch/x86/boot/compressed/efi_mixed.o
  CC      arch/x86/boot/compressed/misc.o
  ZSTD22  arch/x86/boot/compressed/vmlinux.bin.zst
  MKPIGGY arch/x86/boot/compressed/piggy.S
  AS      arch/x86/boot/compressed/piggy.o
  LD      arch/x86/boot/compressed/vmlinux
  ZOFFSET arch/x86/boot/zoffset.h
  OBJCOPY arch/x86/boot/vmlinux.bin
  AS      arch/x86/boot/header.o
  LD      arch/x86/boot/setup.elf
  OBJCOPY arch/x86/boot/setup.bin
  BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready  (#1)
root@laiba:/kernels/linux-6.8#
```

Compiled the kernel image using 4 CPU cores for faster processing.

MAKING A SYSTEM CALL

STEP 1: Go to Kernel Source Folder

Command:

cd /kernels/linux-6.8

```
laiba@laiba:~$ cd /kernels/linux-6.8
laiba@laiba:/kernels/linux-6.8$
```

STEP 2: Create the System Call File

Command:

cd kernel

```
laiba@laiba:/kernels/linux-6.8$ cd kernel
laiba@laiba:/kernels/linux-6.8/kernel$ gedit helloworld.c
```

gedit helloworld.c

```
Open    helloworld.c /kernels/linux-6.8/kernel    Save
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(helloworld)
{
    printk(KERN_INFO "Hello World from Kernel\n");
    return 0;
}
```

At the end this line will be printed.

STEP 3: Add File to Makefile

Command:

Gedit Makefile

```
root@laiba:/kernels/linux-6.8/kernel# gedit Makefile
```

Find line starting with obj-y +=
Add helloworld.o at the end

```
# SPDX-License-Identifier: GPL-2.0
#
# Makefile for the linux kernel.
#

obj-y     = fork.o exec_domain.o panic.o \
            cpu.o exit.o softirq.o resource.o \
            sysctl.o capability.o ptrace.o user.o \
            signal.o sys.o umh.o workqueue.o pid.o task_work.o \
            extable.o params.o \
            kthread.o sys_ni.o nsproxy.o \
            notifier.o ksysfs.o cred.o reboot.o \
            async.o range.o smpboot.o ucount.o regset.o ksyms_common.o\
            helloworld.o

```

This tells the kernel to compile your file.

STEP 4: Register the System Call

Command:

cd ..

Open syscall table:

gedit arch/x86/entry/syscalls/syscall_64.tbl

```
root@laiba:/kernels/linux-6.8/kernel# cd ..
root@laiba:/kernels/linux-6.8# gedit arch/x86/entry/syscalls/syscall_64.tbl
```

Add this line:

462 common helloworld sys_helloworld

```
443     common  quotactl_fd              sys_quotactl_fd
444     common  landlock_create_ruleset sys_landlock_create_ruleset
445     common  landlock_add_rule       sys_landlock_add_rule
446     common  landlock_restrict_self  sys_landlock_restrict_self
447     common  memfd_secret            sys_memfd_secret
448     common  process_mrelease        sys_process_mrelease
449     common  futex_waitv             sys_futex_waitv
450     common  set_mempolicy_home_node sys_set_mempolicy_home_node
451     common  cachestat               sys_cachestat
452     common  fchmodat2               sys_fchmodat2
453     64      map_shadow_stack        sys_map_shadow_stack
454     common  futex_wake              sys_futex_wake
455     common  futex_wait              sys_futex_wait
456     common  futex_requeue           sys_futex_requeue
457     common  statmount               sys_statmount
458     common  listmount               sys_listmount
459     common  lsm_get_self_attr       sys_lsm_get_self_attr
460     common  lsm_set_self_attr       sys_lsm_set_self_attr
461     common  lsm_list_modules        sys_lsm_list_modules
462     common  helloworld              sys_helloworld
#
# Due to a historical design error, certain syscalls are numbered differently
# in x32 as compared to native x86_64.  These syscalls have numbers 512-547.
```

This connects syscall number 548 to your function.

STEP 5: Build the Kernel

Commands:

make -j4 bzImage

```
root@laiba: /kernels/linux-6.8
  AS      arch/x86/boot/compressed/mem_encrypt.o
  CC      arch/x86/boot/compressed/pgtable_64.o
  CC      arch/x86/boot/compressed/sev.o
  CC      arch/x86/boot/compressed/acpi.o
  CC      arch/x86/boot/compressed/tdx.o
  AS      arch/x86/boot/compressed/tdcall.o
  CC      arch/x86/boot/compressed/tdx-shared.o
  CC      arch/x86/boot/compressed/mem.o
  CC      arch/x86/boot/compressed/efi.o
  AS      arch/x86/boot/compressed/efi_mixed.o
  CC      arch/x86/boot/compressed/misc.o
  ZSTD22  arch/x86/boot/compressed/vmlinux.bin.zst
  MKPIGGY arch/x86/boot/compressed/piggy.S
  AS      arch/x86/boot/compressed/piggy.o
  LD      arch/x86/boot/compressed/vmlinux
  ZOFFSET arch/x86/boot/zoffset.h
  OBJCOPY arch/x86/boot/vmlinux.bin
  AS      arch/x86/boot/header.o
  LD      arch/x86/boot/setup.elf
  OBJCOPY arch/x86/boot/setup.bin
  BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready  (#1)
root@laiba:/kernels/linux-6.8#
```

make modules -j4

```
  LD [M]  net/nfc/hci/hci.ko
  LD [M]  net/nfc/nfc_digital.ko
  LD [M]  net/psample/psample.ko
  LD [M]  net/ife/ife.ko
  LD [M]  net/openvswitch/openvswitch.ko
  LD [M]  net/openvswitch/vport-vxlan.ko
  LD [M]  net/openvswitch/vport-geneve.ko
  LD [M]  net/openvswitch/vport-gre.ko
  LD [M]  net/vmw_vsock/vsock.ko
  LD [M]  net/vmw_vsock/vsock_diag.ko
  LD [M]  net/vmw_vsock/vmw_vsock_vmci_transport.ko
  LD [M]  net/vmw_vsock/vmw_vsock_virtio_transport.ko
  LD [M]  net/vmw_vsock/vmw_vsock_virtio_transport_common.ko
  LD [M]  net/vmw_vsock/hv_sock.ko
  LD [M]  net/vmw_vsock/vsock_loopback.ko
  LD [M]  net/nsh/nsh.ko
  LD [M]  net/hsr/hsr.ko
  LD [M]  net/qrtr/qrtr.ko
  LD [M]  net/qrtr/qrtr-smd.ko
  LD [M]  net/qrtr/qrtr-tun.ko
  LD [M]  net/qrtr/qrtr-mhi.ko
  LD [M]  virt/lib/irqbypass.ko
root@laiba:/kernels/linux-6.8#
```

STEP 6: Install the Kernel

Commands:

sudo make modules_install
sudo make install
sudo update-grub

```
root@laiba:/kernels/linux-6.8# sudo make modules_install -j4
  SYMLINK /lib/modules/6.8.0k240812/build
  INSTALL /lib/modules/6.8.0k240812/modules.order
  INSTALL /lib/modules/6.8.0k240812/modules.builtin
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/events/amd/amd-uncore.ko
  INSTALL /lib/modules/6.8.0k240812/modules.builtin.modinfo
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/events/intel/intel-cstate.ko
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/events/rapl.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/events/intel/intel-cstate.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/events/amd/amd-uncore.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/events/rapl.ko
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/kernel/cpu/mce/mce-inject.ko
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/kernel/msr.ko
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/kernel/cpuid.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/kernel/msr.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/kernel/cpu/mce/mce-inject.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/kernel/cpuid.ko
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/crypto/twofish-x86_64.ko
  SIGN    /lib/modules/6.8.0k240812/kernel/arch/x86/crypto/twofish-x86_64.ko
  INSTALL /lib/modules/6.8.0k240812/kernel/arch/x86/crypto/twofish-x86_64-3way.k
```

```
  DEPMOD  /lib/modules/6.8.0k240812
root@laiba:/kernels/linux-6.8# sudo make install -j4
  INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/dkms 6.8.0k240812 /boot/vmlinuz-6.8.
0k240812
 * dkms: running auto installation service for kernel 6.8.0k240812
 * dkms: autoinstall for kernel 6.8.0k240812                              [ OK ]
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.8.0k240812 /boot/v
mlinuz-6.8.0k240812
update-initramfs: Generating /boot/initrd.img-6.8.0k240812
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.8.0k240812 /bo
ot/vmlinuz-6.8.0k240812
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.8.0k240812 /boot/v
mlinuz-6.8.0k240812
run-parts: executing /etc/kernel/postinst.d/vboxadd 6.8.0k240812 /boot/vmlinuz-6
.8.0k240812
VirtualBox Guest Additions: Building the modules for kernel 6.8.0k240812.

VirtualBox Guest Additions: Look at /var/log/vboxadd-setup.log to find out what
went wrong
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.8.0k240812
/boot/vmlinuz-6.8.0k240812
I: /boot/initrd.img is now a symlink to initrd.img-6.8.0k240812
```

Found Linux 6.8.0-k240812 – this one is the custom kernel, we configured.

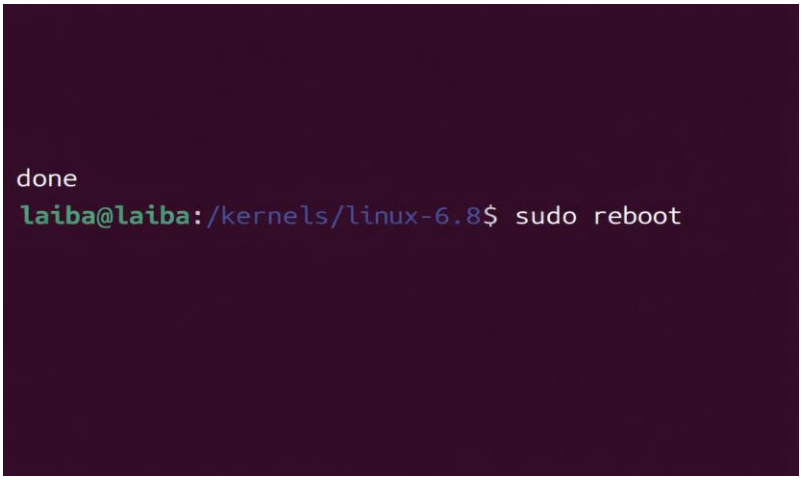
STEP 7: Reboot into Custom Kernel

Command:
sudo reboot

When system starts:

- Hold **Shift**
- Select **Advanced options**

- Choose:

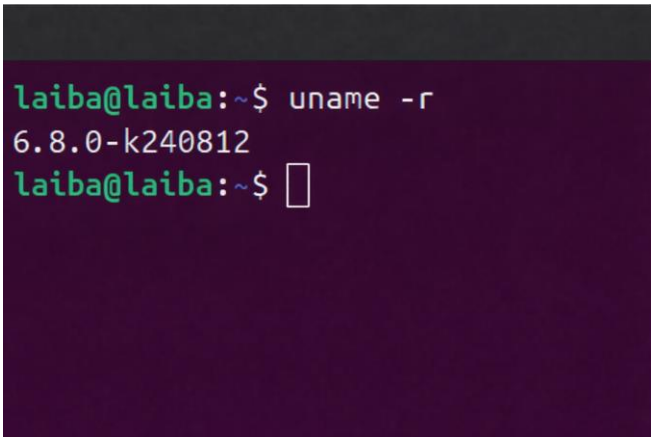


6.8.0-k240812

STEP 8: Confirm You Are in Custom Kernel

Command:

uname -r



STEP 9: Create User Program

Command:

nano test.c

```
laiba@laiba:~$ nano test.c
laiba@laiba:~$
```

```
  GNU nano 7.2                                  test.c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

#define SYS_helloworld 548

int main() {
    long ret = syscall(SYS_helloworld);
    printf("System call returned: %ld\n", ret);
    return 0;
}
```

STEP 10: Compile and Run the Test Program

Command:

gcc test.c -o test
./test

```
laiba@laiba:~$ gcc test.c -o test
laiba@laiba:~$ ./test
System call returned: 0
laiba@laiba:~$
```

STEP 11: Check Kernel Output

Command:

sudo dmesg | tail

```
laiba@laiba:~$
sudo dmesg | tail
[sudo] password for laiba:
[  184.913260] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 32 ti
s, consider switching to WQ_UNBOUND
[  286.556629] systemd-journald[289]: File /var/log/journal/ed2827d5a9c447a186
a6fa2c05573c/user-1000.journal corrupted or uncleanly shut down, renaming and
placing.
[  287.415045] rfkill: input handler enabled
[  297.569769] rfkill: input handler disabled
[  300.729537] ISO 9660 Extensions: Microsoft Joliet Level 3
[  300.734608] ISO 9660 Extensions: RRIP_1991A
[  664.954703] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 64 times,
onsider switching to WQ_UNBOUND
[  787.367210] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 64 ti
s, consider switching to WQ_UNBOUND
[  859.156638] workqueue: ata_sff_pio_task hogged CPU for >10000us 8 times, co
ider switching to WQ_UNBOUND
[ 1082.548574] Hello World from Kernel!
```

The line “Hello World from kernel “ is printed by HelloWorld system call.